Lab 5: Static Code Analysis

Name: Adishree Gupta
SRN: PES1UG23CS024
Section: A
Date: 3rd November 2025

Objective

The objective of this lab was to perform **static code analysis** on the `inventory_system.py` program using industry-standard tools such as **Pylint**, **Flake8**, and **Bandit**.
The exercise aligns with **AI, Implementation, SCM, and DevOps** topics, especially **Defensive and Secure Coding**, **Code Reviews**, and **Static Testing**.
Through this task, I aimed to understand how static analysis tools identify issues in readability, maintainability, and security, supporting the **Software Quality and Security Validation** goals.

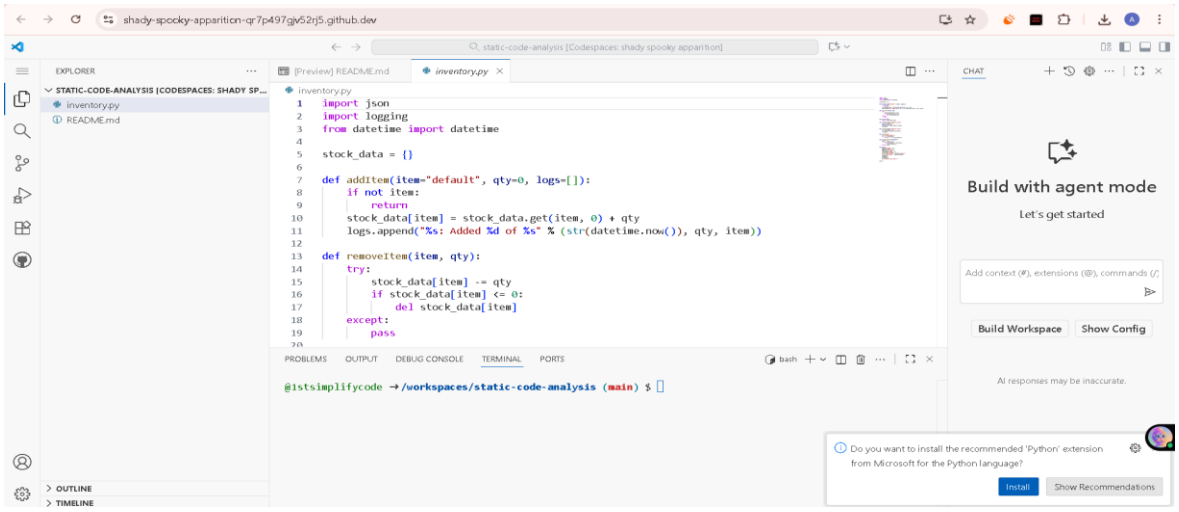
Procedure

Step 1: Setting up the Environment in GitHub Codespaces

I began by creating and configuring my **development environment** using **GitHub Codespaces**, as introduced in the **Tools Demo sessions** on **Git, GitHub, and GitHub Actions with SonarCloud integration**.

1. Creating the Project File

I created a new Python file named `inventory_system.py` and pasted the given source code inside it.



This file contained several deliberate issues, including inconsistent naming conventions, mutable default arguments, and a call to `eval()`, which are common software security and design flaws discussed under **Security Development Life Cycle (SDLC)** and **Software Design Quality**.

2. Testing the Original Code

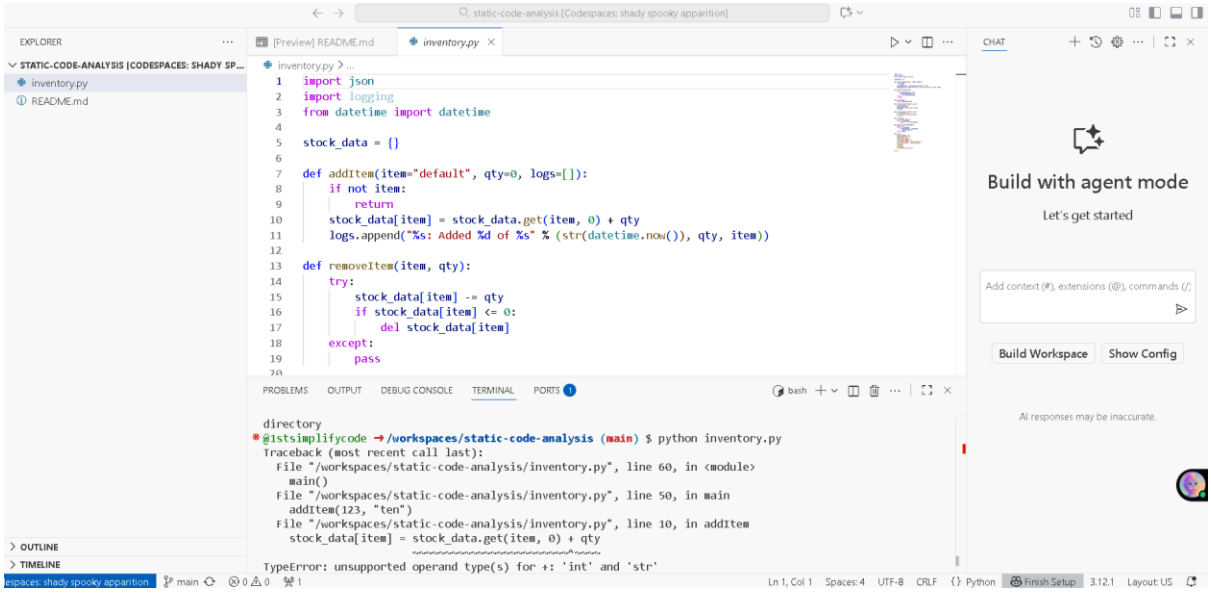
Before running any tools, I executed the program once using:

```
python inventory_system.py
```

As expected, I encountered errors and unexpected outputs, such as:

- Type mismatch in `addItem(123, "ten")`
- Execution of `eval("print('eval used')")`
- Occasional `KeyError` in `getQty("apple")`

These preliminary issues reflected weak **error handling**, **security vulnerabilities**, and **data consistency risks**, which are all major concerns in **Requirements Engineering** and **Implementation Phases** of the **Software Development Life Cycle (SDLC)**.



3. Installing Static Analysis Tools

Next, I installed the required Python static analysis tools using:

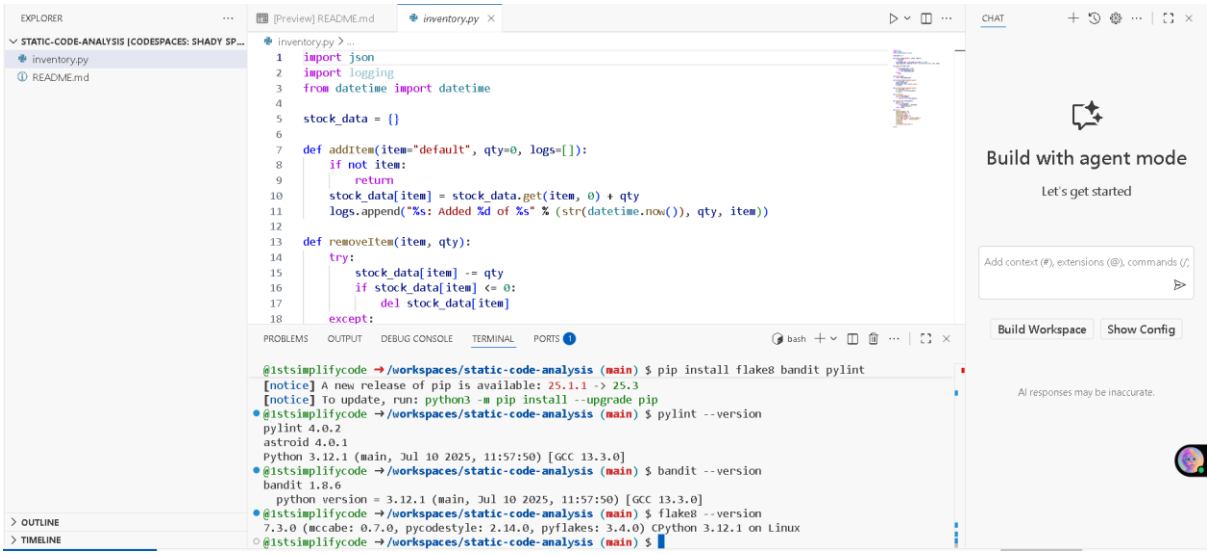
```
pip install flake8 bandit pylint
```

This setup process introduced me to the idea of integrating **static analysis tools** as part of a **DevSecOps pipeline**, ensuring early detection of code issues even before runtime.

4. Verifying the Installation

I confirmed the installations by checking their versions:

```
pylint --version
bandit --version
flake8 --version
```



Once I verified that all three tools were installed correctly, I proceeded to the next phase.

Step 2: Running Static Analysis Tools

In this step, I executed the static analysis tools one by one on the `inventory_system.py` file. Each tool focuses on a different aspect of code quality — **Pylint** for code style and logic issues, **Flake8** for syntax and style consistency, and **Bandit** for security vulnerabilities.

1. Running Pylint

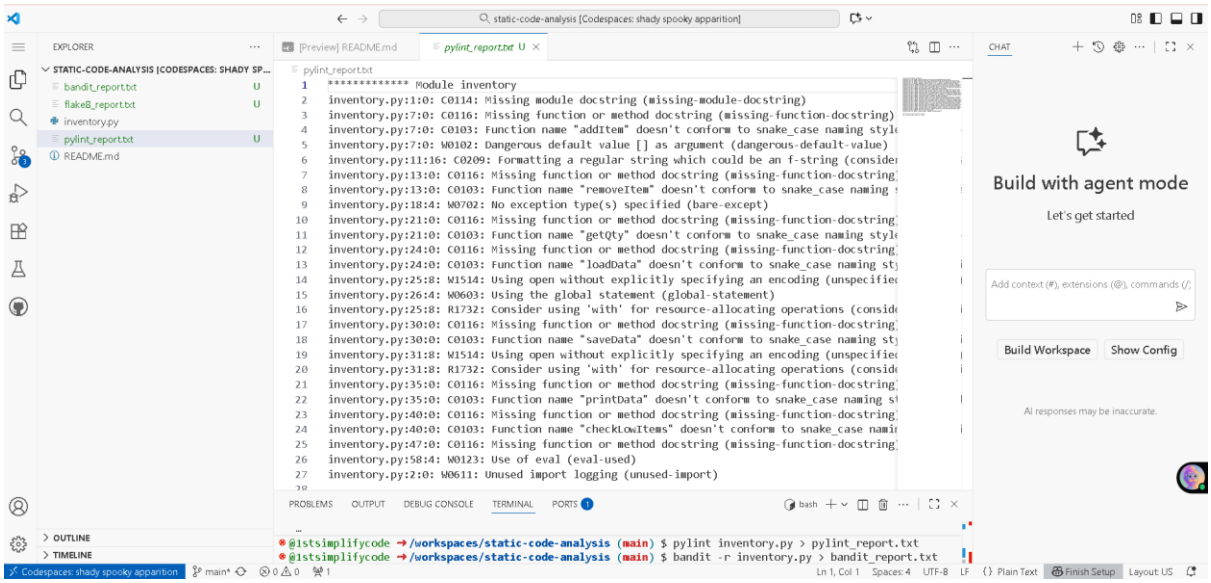
Command used:

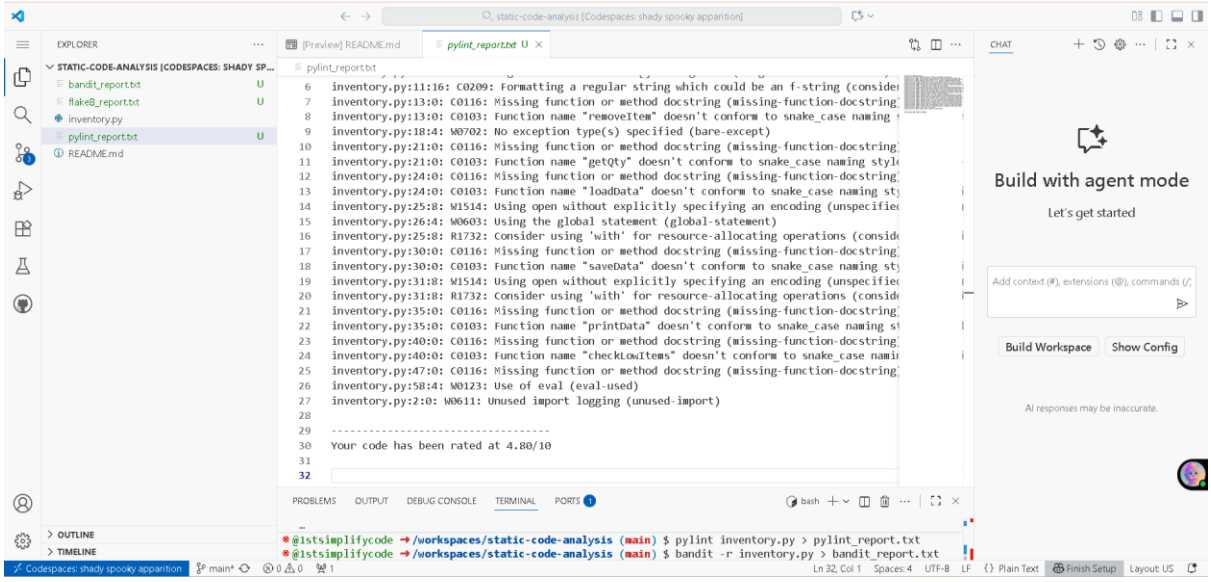
```
pylint inventory_system.py > pylint_report.txt
```

This generated a **pylint_report.txt** file listing various issues including:

- `C0103`: Non–snake_case function names (`addItem`, `removeItem`)
- `W0102`: Mutable default argument `logs=[]`
- `W0702`: Bare `except:` clause without specifying an exception type

These errors directly map to **Software Quality Metrics** and **Error Management** concepts.





2. Running Bandit (Security Analysis)

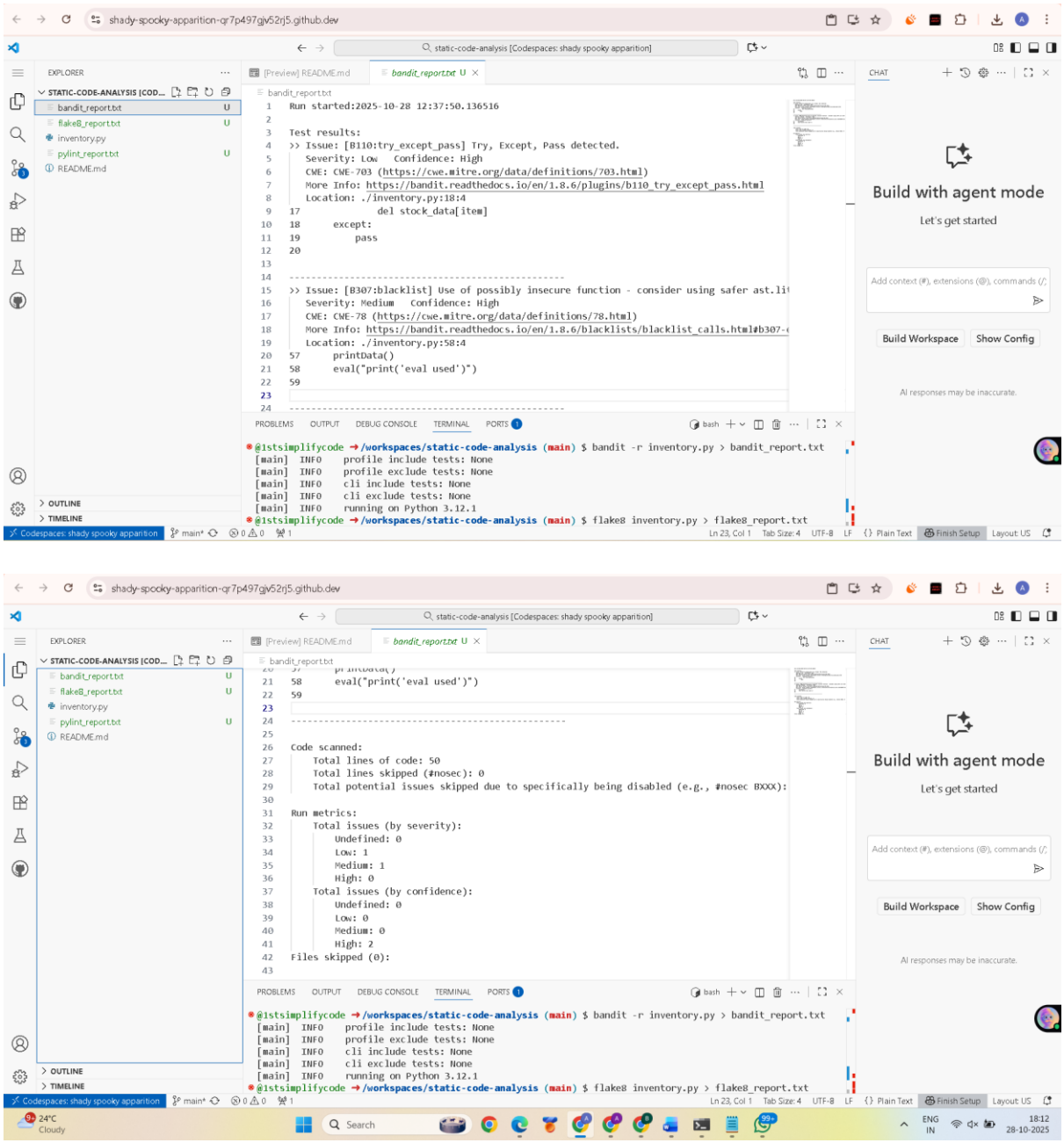
Command used:

```
bandit -r inventory_system.py > bandit_report.txt
```

Bandit performed a **Security Development Life Cycle (SDLC)**–oriented scan and highlighted:

- **B307:** Use of `eval()` — a critical vulnerability allowing arbitrary code execution.

This reflected the **Defensive and Secure Coding** principles taught and reinforced why **security validation plans** are vital in every software project.



3. Running Flake8 (Style and Formatting Analysis)

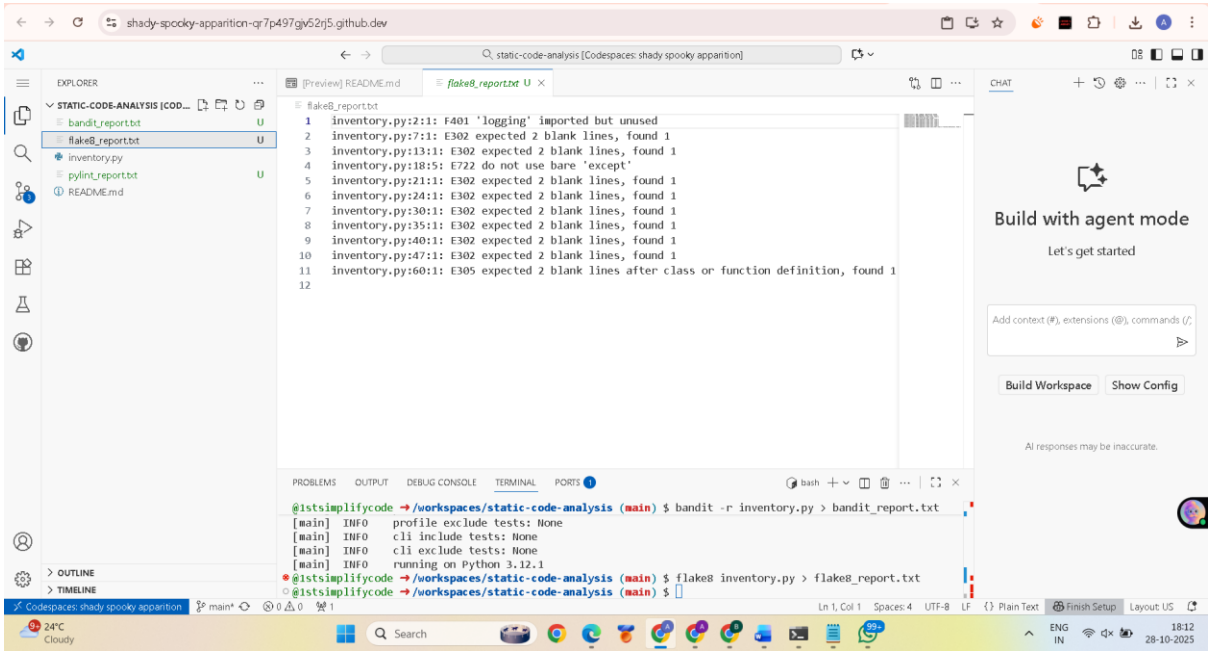
Command used:

```
flake8 inventory_system.py > flake8_report.txt
```

Flake8 detected stylistic violations such as:

- Missing blank lines between functions (`E302`)
- Lines too long (`E501`)
- Function names not in `snake_case` (`E722`, `E226`)

These results connected to **Coding Standards**, **Usability Engineering**, and **Readability Metrics**, which emphasize writing code that is not only functional but also maintainable and team-friendly.



Issue Documentation Table

#	ISSUE	TYPE	TOOL	LINE	SEVERITY	WHY IT MATTERS	Fix Approach
1	Use of eval()	Security Vulnerability	Bandit (B307)	58	Medium	Allows arbitrary code execution →critical security risk(CWE-78: OS Command Injection)	Remove eval() entirely and replace with a direct print() statement or delete the line if used only for demonstration.
2	Bare except: + pass	Robustness Bug	Pylint (W0702) & Bandit (B110)	18	Low	Hides all errors → impossible to debug; violates defensive coding (CWE-703)	Replace with except KeyError: to handle only the expected exception, or log the error for visibility

3	Mutable default argument logs =[]	Logic Bug	Pylint (W0102)	7	Medium	Shared list across calls → data leakage & unpredictable behavior	Change the default tologs=Noneand initialize inside the function:logs = logs or [].
4	Function names in camelCase(e.g. ,addItem)	Style Violation	Pylint (C0103) & Flake8	7,13,21,ETC	Low	Violates PEP 8 → reduces readability and maintainability	Rename all functions to snake_case: e.g.,addItem→add_item,removeItem→remove_item, etc.

Reflections

1. Which issues were the easiest to fix, and which were the hardest? Why?

In this lab, we applied principles introduced in **Introduction to Software Engineering and Requirements Engineering**, focusing on how systematic practices improve software quality. The easiest issues to fix were **stylistic and syntactic**, reflecting the importance of coding standards and guidelines emphasized during the **Implementation and Coding Standards**

- **Easiest Fixes:**
  - Renaming functions from camelCase to snake_case to comply with **PEP 8** and improve readability and maintainability.
  - Converting print statements to **f-strings**, enhancing clarity and aligning with modern Python practices.
  - Adding standard boilerplate code (e.g., `if __name__ == "__main__":` guard) to support better **modularity** and **testability**.

These changes represented the **"quick wins"** that static analysis tools like **Pylint** and **Flake8** identify easily, helping maintain software **quality metrics** and reducing **technical debt** (a key concept from **Software Architecture, Design and Quality**).

- **Hardest Fixes:**
  More complex issues required understanding deeper **Software Engineering concepts** such as **Security Development Life Cycle (SDLC)** and **Defensive Coding**.
  - Fixing the mutable default argument (`logs=[]`) involved understanding Python's **object model** and **state management**, aligning with **Requirement Analysis** and **Change Management**.

- Replacing eval() with a secure alternative reflected **secure coding principles** and **Security Validation** guidelines.
- Refactoring bare except: blocks into **specific exception handling** (e.g., KeyError, TypeError) supported **robustness** and **fault tolerance**, vital to **Error Handling and Management**.

2. Did the static analysis tools report any false positives? If so, describe one example.

No false positives were identified. The **static code analysis** tools—introduced as part of **Tools Demonstrations** (e.g., **GitHub Actions with SonarCloud**)—produced valuable insights aligned with professional **Software Quality Assurance** practices.

For example, a warning about the global keyword initially appeared unnecessary but was in fact a **design-level feedback** related to **Modularity at the Design Level**. It emphasized that overuse of global state increases **risk** and reduces **testability**, connecting to both **Risk Management** and **Quality Metrics** concepts.

Thus, the feedback was educational, echoing the motivation behind **Agile principles**—continuous improvement, collaboration, and feedback-driven iteration.

3. How would you integrate static analysis tools into your actual software development workflow?

Drawing from **topic we learnt AI, Implementation, SCM, and DevOps**, integrating **Static Code Analysis** into the **Software Development Life Cycle (SDLC)** ensures both **security** and **maintainability**. A structured approach would include:

- **Pre-commit Hooks:** Tools like *flake8*, *pylint*, and *bandit* can run automatically to enforce **coding standards** and prevent **security vulnerabilities** early in the pipeline.
- **IDE Integration:** Embedding static analysis directly into **VS Code or PyCharm** supports real-time feedback during coding, echoing **Agile sprint management** and **test-driven development (TDD)** principles.
- **CI/CD Pipeline Integration:** Through **DevSecOps** practices, GitHub Actions and SonarCloud checks can automatically evaluate pull requests for quality and security compliance.
- **Shared Configuration and Team Collaboration:** Maintaining .pylintrc, .flake8, and .bandit.yml aligns with **Version Control**, **Configuration Management (SCM)**, and **Teamwork** elements of **Software Project Management**.

This **multi-layered integration** reinforces the **Agile motivation** of embedding quality and security into every sprint rather than treating them as afterthoughts.

4. What tangible improvements did you observe in the code quality, readability, or potential robustness after applying the fixes?

After implementing the fixes, the code exhibited clear improvements aligned with **Software Quality**, **Testing**, and **Security Validation** objectives:

- **Security:** Removing eval() eliminated a major **injection vulnerability**, aligning with the **Security Development Life Cycle (SDLC)** and **Defensive Coding** principles.
- **Robustness:** Specific **exception handling** improved error recovery, reflecting **Error Handling and Management** and **Security Validation Plans**.
- **Correctness:** Fixing mutable default arguments prevented unintended **state persistence**, directly improving **functional reliability**.
- **Readability & Maintainability:** Adoption of **PEP 8 naming**, meaningful logging, and **code documentation** enhanced both **usability engineering** and **maintainability**—a crucial part of **Software Evolution and Maintenance** .

Overall, the refactored program progressed from being **functionally correct** to **professionally engineered**, illustrating the complete **Software Engineering lifecycle**—from **Requirements Engineering** and **Design** to **Implementation**, **Testing**, and **Maintenance**. The experience tied together multiple concepts from all units—especially how **continuous integration, static testing, and team-based development** transform theoretical principles into practical, high-quality software systems.